Chapter 1 Trees and Ensemble Learning

Introduction

- ❑ *Decision Trees and Axis-Aligned Splits*
- ❑ *Information Gain and Mutual-Information*
- ❑ *Feature Selection and Purity Measures*
- ❑ *Bagging and Variance Reduction*
- ❑ *Random Forests and Out-of-Bag Evaluation*
- ❑ *Boosting and Functional Gradient View*

Review: Gaussian Processes and Bayesian Optimization

In the previous chapter, we introduced Gaussian Processes (GPs) and showed how they can be used for regression. Given training inputs

$$X = (x_1, \dots, x_n), \qquad y = (y_1, \dots, y_n)^\top, \tag{1.1}$$

and a test point $x_\star$, a GP prior with kernel $k(\cdot,\cdot)$ and i.i.d. Gaussian observation noise of variance λ yields the joint Gaussian distribution

$$\begin{pmatrix} y \\ f_\star \end{pmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K(X,X) + \lambda I_n & k(X, x_\star) \\ k(x_\star, X) & k(x_\star, x_\star) \end{bmatrix}\right), \tag{1.2}$$

where $K(X,X)$ is the $n \times n$ Gram matrix and $k(X, x_\star) = (k(x_1, x_\star), \dots, k(x_n, x_\star))^\top$. Conditioning on the observed y gives the posterior predictive distribution for $f_\star = f(x_\star)$:

$$\mu_\star = \mathbb{E}[f_\star \mid X, y, x_\star] = k(x_\star, X)^\top \big(K(X,X) + \lambda I_n\big)^{-1} y, \tag{1.3}$$

$$\sigma_\star^2 = \mathrm{Var}[f_\star \mid X, y, x_\star] = k(x_\star, x_\star) - k(x_\star, X)^\top \big(K(X,X) + \lambda I_n\big)^{-1} k(x_\star, X). \tag{1.4}$$

By evaluating $\mu_\star$ at many test points and optionally plotting $\mu_\star \pm \sigma_\star$, we obtain a smooth predictive curve together with credible intervals that quantify uncertainty.

Black-Box Optimization with Gaussian Processes

An important application of GP regression is *Bayesian Optimization* (BO), which addresses *black-box optimization* problems:

$$\operatorname*{argmax}_{x \in \mathcal{X}} f(x), \tag{1.5}$$

where:

- $f(x)$ has no known analytic form (no closed-form expression);
- gradients $\nabla f(x)$ are unavailable;
- each evaluation of $f(x)$ is *expensive*.

We can, however, query the black box at chosen points:

$$y = f(x) + \varepsilon, \tag{1.6}$$

and use these point evaluations to build a surrogate model.

A canonical example is *hyperparameter tuning* for neural networks or other machine learning models. Here:

- x encodes a vector of hyperparameters (learning rate, width, depth, regularization strength, …);
- $f(x)$ is the validation performance (e.g. accuracy) obtained by training the model with hyperparameters x and evaluating on a validation set.

Each function evaluation requires a full train–validate cycle and is therefore very costly, so we must find good x using as *few* evaluations as possible.

Remark

Bayesian Optimization with a GP surrogate proceeds iteratively:

1. ***Initialization.*** *Select an initial design* $\{x_i\}_{i=1}^n$ *(e.g. random or space-filling) and evaluate* $y_i = f(x_i)$.
2. ***Fit GP surrogate.*** *Use* $\{(x_i, y_i)\}_{i=1}^n$ *to fit a GP regression model, yielding posterior mean* $\mu(x)$ *and variance* $\sigma^2(x)$ *for all* $x \in \mathcal{X}$.
3. ***Acquisition maximization.*** *Define an* acquisition function $a(x)$ *that uses* $\mu(x)$ *and* $\sigma(x)$ *to score candidate points, and choose the next evaluation point(s) by*
$$x_{next} \in \operatorname*{argmax}_{x \in \mathcal{X}} a(x). \tag{1.7}$$
4. ***Evaluate and update.*** *Query the black box at* x_{next} *to obtain* $y_{next} = f(x_{next})$*, augment the dataset, and refit (or update) the GP surrogate.*
5. ***Repeat*** *Steps 2–4 until a pre-specified evaluation budget is exhausted, then return the best observed* y *(or its maximizer).*

The acquisition function is responsible for balancing exploration *(sampling uncertain regions where* $\sigma(x)$ *is large) and* exploitation *(sampling near currently promising regions where* $\mu(x)$ *is large).*

Acquisition Functions

Let $y_{\max}$ denote the best function value observed so far, and suppose the GP posterior at x is Gaussian:
$$f(x) \mid \mathcal{D} \sim \mathcal{N}\big(\mu(x), \sigma^2(x)\big). \tag{1.8}$$

Two widely used acquisition functions are:

- **Expected Improvement (EI).** Define the *improvement* at x as
$$I(x) = \big(f(x) - y_{\max}\big)_+ = \max\{f(x) - y_{\max}, 0\}. \tag{1.9}$$
The Expected Improvement is
$$a_{\mathrm{EI}}(x) = \mathbb{E}[I(x) \mid \mathcal{D}]. \tag{1.10}$$
Under the Gaussian posterior, this has a closed form:
$$a_{\mathrm{EI}}(x) = (\mu(x) - y_{\max})\Phi(z) + \sigma(x)\phi(z), \qquad z = \frac{\mu(x) - y_{\max}}{\sigma(x)}, \tag{1.11}$$
where Φ and ϕ are the CDF and PDF of the standard normal distribution, respectively. EI focuses on regions that, on average, are likely to yield improvements over the current best value.
- **Upper Confidence Bound (UCB).** For a parameter $\kappa > 0$, define
$$a_{\mathrm{UCB}}(x) = \mu(x) + \kappa\,\sigma(x). \tag{1.12}$$
When $\kappa = 1$, $a_{\mathrm{UCB}}(x)$ follows the upper envelope $\mu(x) + \sigma(x)$ of the GP posterior. Larger κ encourages more exploration (sampling high-uncertainty regions), while smaller κ favors exploitation.

In practice, BO alternates between refining the surrogate in regions where data are scarce and zooming in on promising areas, progressively improving our estimate of both the location and value of the optimum.

Remark

Many modern hyperparameter optimization libraries implement Bayesian Optimization with GP or related surrogates. For example, `Optuna` *provides a flexible interface where the user specifies:*

- *the search space of hyperparameters;*

- *an evaluation budget (maximum number of trials);*
- *an objective function that trains and evaluates the model.*

The library then automatically manages the BO loop, returning (approximately) optimal hyperparameters within the given budget.

1.1 Decision Trees and Feature Selection

1.1.1 Decision Trees and Axis-Aligned Splits

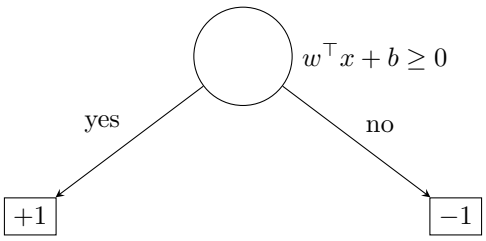
Previously, we used linear models (or linear models in a high-dimensional feature space) to perform binary classification. The classifier can be written as

$$f(x) = \begin{cases} +1, & \text{if } w^\top x + b \geq 0, \\ -1, & \text{if } w^\top x + b < 0. \end{cases} \tag{1.13}$$

The decision boundary is the hyperplane

$$w^\top x + b = 0. \tag{1.14}$$

Another way to obtain more flexible, nonlinear decision boundaries is to use *tree-based models*. A simple decision tree implementing the rule above can be drawn as follows:



Definition 1.1 (Decision Tree)

A decision tree *is a tree consisting of a root node, internal nodes, leaf nodes, and directed edges. Each internal node partitions the data according to some feature. For any input example, its final prediction is given by the label of the leaf node it reaches.* ♣

Example 1.1

We consider a binary classification setting where the label set is

$$Y \in \{-1, +1\}. \tag{1.15}$$

Here, the interpretation is:

$$+1: \text{ a good researcher}, \qquad -1: \text{ a bad researcher}. \tag{1.16}$$

Assume we have three binary features:

- Feature A:

$$+1: \text{ hardworking}, \qquad -1: \text{ not hardworking}. \tag{1.17}$$

- Feature B:

$$+1: \text{ has good vision}, \qquad -1: \text{ not}. \tag{1.18}$$

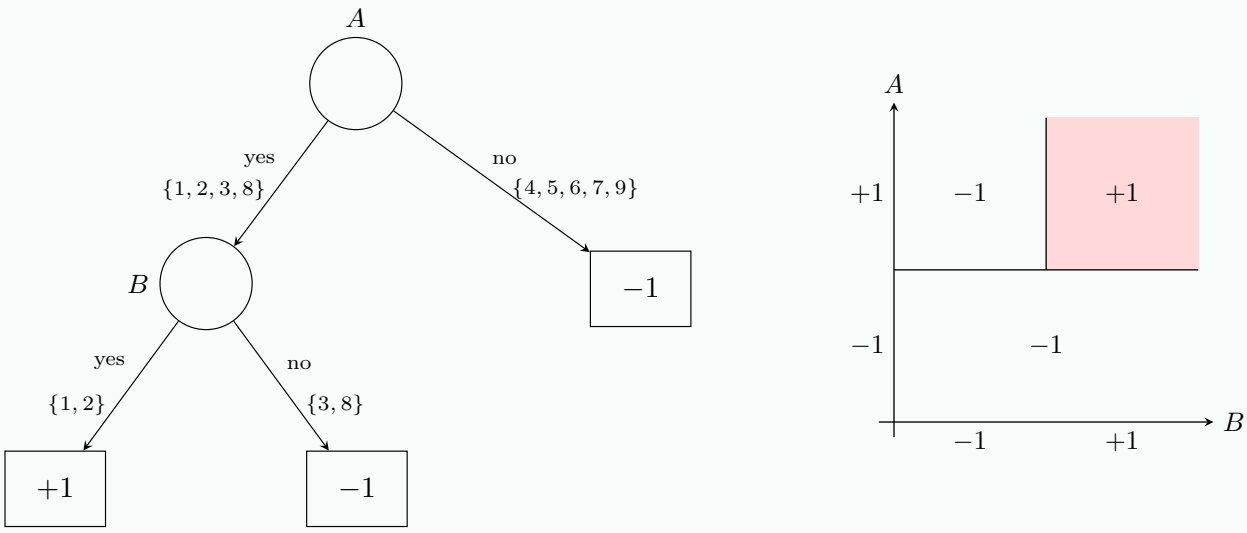
ID	A	B	C	y
1	✓	✓	✓	✓
2	✓	✗	✗	✓
3	✓	✗	✓	✗
4	✗	✓	✓	✗
5	✗	✗	✗	✗
6	✗	✗	✗	✗
7	✗	✓	✗	✗
8	✓	✗	✓	✗
9	✗	✓	✓	✗

- Feature C:

$$+1: \text{ likes bananas}, \qquad -1: \text{ not}. \tag{1.19}$$

A decision tree may use these features at its internal nodes to determine whether an example will ultimately be predicted as $+1$ or -1.

Solution



If we choose feature C as the root node, the samples are not separated as cleanly as when we use feature A. This leads to a natural question: *given many candidate features, how can we choose the one that produces the "purest" split at the root?*

From the example above, a good feature should split the training set into subsets whose labels are as pure (i.e., as close to all positive or all negative) as possible. To quantify this notion of purity—or, conversely, of randomness—we introduce the notion of entropy.

1.1.2 Entropy, Information Gain, and Mutual Information

Definition 1.2 (Entropy)

For a discrete random variable X with distribution P, the (Shannon) entropy of P is defined as

$$H(P) := -\sum_x P(x) \log P(x). \tag{1.20}$$

Entropy measures the intrinsic uncertainty of the distribution. Equivalently,

$$H(P) = \mathbb{E}_{X\sim P}\big[-\log P(X)\big], \tag{1.21}$$

so $-\log P(X)$ can be viewed as the information content *of the outcome X, and $H(P)$ is its average over many draws.* ♣

Entropy has several essential properties.

1. **Non-negativity.** Since $\log P(x) \le 0$ for all x, we have

$$H(P) = -\sum_x P(x) \log P(x) \ge 0, \tag{1.22}$$

with equality if and only if the distribution is degenerate (i.e., $P(x) = 1$ for some x).

2. **Upper bound.** Using Jensen's inequality:

$$H(P) = -\mathbb{E}_P[\log P(X)] \leq -\log \mathbb{E}_P[P(X)] = -\log \sum_x P(x)^2. \tag{1.23}$$

In particular, for a distribution supported on n distinct outcomes,

$$H(P) \leq \log n, \tag{1.24}$$

and equality holds if and only if P is the uniform distribution:

$$P(x) = \frac{1}{n}. \tag{1.25}$$

Remark

Beyond measuring uncertainty, entropy also characterizes the fundamental limit of lossless data compression. For a discrete source with distribution P, any prefix-free coding scheme assigns a codeword of length $\ell(x)$ to each symbol x. Shannon's source coding theorem states that the expected code length satisfies

$$\mathbb{E}[\ell(X)] \;\geq\; H(P) := -\sum_x P(x) \log P(x), \tag{1.26}$$

with equality achievable asymptotically.
Thus, the entropy $H(P)$ represents the minimal achievable average number of bits required to encode samples drawn from P. In this sense, entropy quantifies both the intrinsic uncertainty of a distribution and the optimal compression rate for a lossless encoding.

Definition 1.3 (Cross-Entropy)

Let P and Q be two distributions over the same support. The cross-entropy between P and Q is defined as

$$H(P, Q) := -\sum_x P(x) \log Q(x) = \mathbb{E}_P[-\log Q(X)]. \tag{1.27}$$

♣

Observe:

$$H(P, Q) - H(P) = -\sum_x P(x) \log Q(x) + \sum_x P(x) \log P(x) = \sum_x P(x) \log \frac{P(x)}{Q(x)}. \tag{1.28}$$

Definition 1.4 (Kullback–Leibler (KL) Divergence)

Let P and Q be two probability distributions over the same support. The Kullback–Leibler divergence from P to Q is defined as

$$D_{\mathrm{KL}}(P \,\|\, Q) := \sum_x P(x) \log \frac{P(x)}{Q(x)} \tag{1.29}$$

in the discrete case, or

$$D_{\mathrm{KL}}(P \,\|\, Q) := \int p(x) \log \frac{p(x)}{q(x)} \, \mathrm{d}x \tag{1.30}$$

in the continuous case.
KL divergence measures how different the distribution P is from Q. It satisfies

$$D_{\mathrm{KL}}(P \| Q) \geq 0, \tag{1.31}$$

with equality if and only if $P = Q$ almost everywhere.

♣

Thus,

$$H(P, Q) - H(P) = D_{\mathrm{KL}}(P \| Q) \geq 0, \tag{1.32}$$

with equality if and only if $P = Q$.

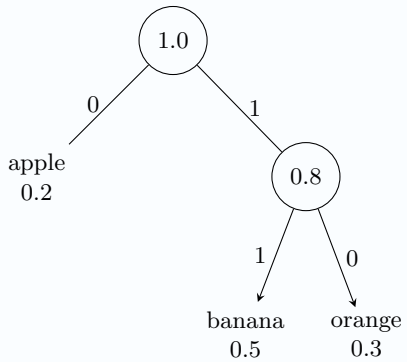
Cross-entropy decomposes into:

$$H(P, Q) = H(P) + D_{\mathrm{KL}}(P\|Q), \tag{1.33}$$

meaning that cross-entropy is always at least as large as entropy and grows with the divergence between P and Q.

Remark

Huffman coding is a greedy algorithm for constructing an optimal prefix-free code when the symbol probabilities are known. More frequent symbols receive shorter codewords and rare symbols receive longer ones. Shannon's source coding theorem guarantees that the expected code length of a Huffman code is no larger than $H(P) + 1$ bits per symbol, so Huffman coding is essentially optimal among all lossless prefix codes.



Huffman coding is tightly connected to entropy and cross-entropy.

1. Entropy as the optimal code length. *For a source with distribution P, the entropy*

$$H(P) = -\sum_x P(x) \log P(x) \tag{1.34}$$

is the theoretical lower bound on the average number of bits needed to encode symbols drawn from P. Huffman coding produces the optimal prefix-free code, and its expected code length L_{Huff} satisfies

$$H(P) \le L_{\mathrm{Huff}} < H(P) + 1. \tag{1.35}$$

2. Cross-entropy as the cost of using a wrong code. *Suppose we design a Huffman code based on a* wrong *distribution Q but the true data come from P. Then the expected code length becomes*

$$\mathbb{E}_P[-\log Q(X)] = H(P, Q), \tag{1.36}$$

which is the cross-entropy between P and Q.

3. Extra penalty equals KL divergence. *The additional number of bits required due to the mismatch between P and Q is*

$$H(P, Q) - H(P) = D_{\mathrm{KL}}(P\|Q). \tag{1.37}$$

Thus, the KL divergence quantifies the inefficiency of using a code optimized for Q when the true distribution is P.

Summary. *Entropy gives the minimal achievable rate, Huffman coding attains it (up to one bit), and cross-entropy / KL divergence measure how much extra cost arises when the coding distribution does not match the true distribution.*

Theorem 1.1 (Shannon's Source Coding Theorem*)

Let X be a discrete memoryless source with distribution P. For any prefix-free code with codeword lengths $\{\ell(x)\}$, the expected code length satisfies

$$\mathbb{E}[\ell(X)] \;\ge\; H(P). \tag{1.38}$$

Moreover, for every $\varepsilon > 0$, there exists a prefix-free code such that

$$\mathbb{E}[\ell(X)] \;\le\; H(P) + \varepsilon, \tag{1.39}$$

for sufficiently long block coding. Thus $H(P)$ is the optimal achievable rate for lossless compression. ♡

Proof

The theorem consists of two parts: a converse (no code beats entropy) and an achievability (entropy can be approached arbitrarily closely).

1. Converse: No prefix-free code can beat $H(P)$. *For any prefix-free code, Kraft's inequality gives*

$$\sum_x 2^{-\ell(x)} \leq 1. \tag{1.40}$$

Multiplying both sides by $P(x)$ and applying Jensen's inequality to the convex function $-\log(\cdot)$ yields

$$\mathbb{E}[\ell(X)] \;=\; \sum_x P(x)\ell(x) \;\geq\; -\sum_x P(x)\log P(x) \;=\; H(P). \tag{1.41}$$

Thus entropy is a lower bound on any valid coding scheme.

2. Achievability: Constructing codes approaching $H(P)$. *Consider n i.i.d. samples X^n from the source. The Asymptotic Equipartition Property (AEP) states that for any $\varepsilon > 0$, with probability approaching 1, sequences in the typical set $\mathcal{T}_\varepsilon^{(n)}$ satisfy*

$$-\frac{1}{n}\log P(X^n) \in [H(P) - \varepsilon,\; H(P) + \varepsilon]. \tag{1.42}$$

The typical set contains approximately $2^{nH(P)}$ sequences. We can therefore assign to each typical sequence a codeword of length

$$nH(P) + o(n),$$

while atypical sequences receive longer codewords but contribute negligibly to the expected length. This yields a block code with

$$\mathbb{E}[\ell(X^n)] \leq n(H(P) + \varepsilon). \tag{1.43}$$

3. Conclusion. *Combining the converse and achievability gives the optimal rate:*

$$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}[\ell(X^n)] = H(P).$$

Thus $H(P)$ is both a fundamental lower bound and an achievable rate for lossless source coding. ■

To measure how "pure" the labels in a dataset are, we use the notion of *information gain*.

Let D be a training set. Labels lie in a finite set $Y \in \{1, \ldots, K\}$. For each label k, define the subset

$$C_k := \{x \in D : y(x) = k\}. \tag{1.44}$$

The entropy of D with respect to its empirical label distribution is

$$H(D) = -\sum_{k=1}^{K} \frac{|C_k|}{|D|}\log\frac{|C_k|}{|D|} \approx -\sum_{y=1}^{K} P(y)\log P(y) = H(Y), \tag{1.45}$$

where $P(y)$ denotes the empirical frequency of label y.

Let A be a feature taking values in $\{a_1, \ldots, a_m\}$, and let

$$D_{a_i} := \{x \in D : A(x) = a_i\} \tag{1.46}$$

be the subset of samples with feature value a_i.

The conditional entropy of labels given the feature is

$$H(D \mid A) = \sum_{i=1}^{m} \frac{|D_{a_i}|}{|D|} H(D_{a_i}) = \sum_{i=1}^{m} \frac{|D_{a_i}|}{|D|}\left(-\sum_{k=1}^{K} \frac{|D_{a_i} \cap C_k|}{|D_{a_i}|}\log\frac{|D_{a_i} \cap C_k|}{|D_{a_i}|}\right), \tag{1.47}$$

Note

$$H(D \mid A) = \mathbb{E}_{x|A}[H(D_x)] \tag{1.48}$$

The *information gain* of feature A is defined as

Definition 1.5 (Information gain (Mutual Information))

$$g(D, A) = H(D) - H(D \mid A). \tag{1.49}$$

♣

A large value of $g(D, A)$ means that feature A significantly reduces label uncertainty, producing purer label subsets and therefore acting as a good splitting feature in decision tree construction.

More generally, for two random variables X and Y, their *mutual information* is

$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X,Y). \tag{1.50}$$

In our setting, information gain is exactly the mutual information between the feature and the label:

$$g(D, A) \approx I(Y;A). \tag{1.51}$$

We therefore choose the best feature A by

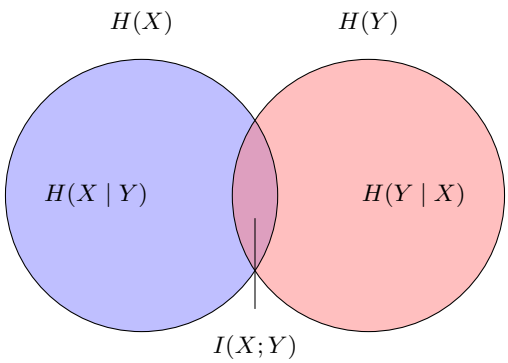
$$A^\star = \operatorname*{argmax}_A g(D, A). \tag{1.52}$$

If two random variables satisfy $I(X;Y) = 0$, then they are independent: knowing one of them does not reduce the uncertainty about the other.

Mutual information has several important properties:

1. **Symmetry.** $I(X;Y) = I(Y;X)$.
2. **Non-negativity.** $I(X;Y) \geq 0$, with equality if and only if X and Y are independent.
3. **Alternative forms.** $I(X;Y) = H(X) + H(Y) - H(X,Y)$.

The larger the mutual information, the more correlated X and Y are. When X and Y are highly correlated, knowing X significantly reduces the uncertainty about Y, and vice versa.



1.1.3 Information Gain Ratio

Information gain has a significant drawback: it tends to favor features with many possible values.

Example 1.2

Consider a dataset with two features: A (taking values a_1, a_2) and B (taking values $b_1, \ldots, b_{10}$). Suppose that:

- Feature A splits the data into two subsets, each containing 5 classes uniformly.
- Feature B splits the data into 10 pure subsets, each containing only one class.

Then $g(D, A) = \log 10 - \log 5 = \log 2$, while $g(D, B) = \log 10 - 0 = \log 10$. Information gain would prefer feature B, even though it may have poor generalization ability.

More extremely, if we use the index (row number) as a feature, each example would fall into its own pure subset, yielding maximum information gain $g(D, \text{index}) = H(D)$, but such a feature has no generalization ability for new test examples.

To address this issue, we introduce the *information gain ratio*, which penalizes features with many values.

Definition 1.6 (Information Gain Ratio)

The information gain ratio of feature A is defined as

$$\mathrm{GR}(D, A) = \frac{g(D, A)}{H(D, A)}, \tag{1.53}$$

where $H(D, A)$ is the entropy of feature A itself:

$$H(D, A) = -\sum_{i=1}^{m} \frac{|D_{a_i}|}{|D|} \log \frac{|D_{a_i}|}{|D|}. \tag{1.54}$$

This measures the intrinsic uncertainty of the feature A's distribution. ♣

The denominator $H(D, A)$ penalizes features with many values. When A has many uniformly distributed values, $H(D, A)$ approaches $\log m$, which reduces the gain ratio. This helps prevent overfitting to features that achieve high purity simply by having many possible values.

1.1.4 Gini Index

Another common purity measure is the *Gini index*, which serves as an alternative to entropy.

Definition 1.7 (Gini Index)

For a dataset D with labels in $\{1, \ldots, K\}$, the Gini index is defined as

$$\mathrm{Gini}(D) = 1 - \sum_{k=1}^{K} \left(\frac{|C_k|}{|D|} \right)^2 = \sum_{k=1}^{K} \frac{|C_k|}{|D|} \left(1 - \frac{|C_k|}{|D|} \right), \tag{1.55}$$

where $C_k = \{x \in D : y(x) = k\}$. ♣

The Gini index measures the expected error rate if we randomly label examples according to the class distribution. It satisfies:

- $\mathrm{Gini}(D) = 0$ when D is pure (all examples have the same label).
- $\mathrm{Gini}(D)$ is maximized when the class distribution is uniform, taking the value $1 - 1/K$ for K classes.
- For binary classification ($K = 2$), $\mathrm{Gini}(D) = 2p(1 - p)$, where p is the proportion of the positive class.

The conditional Gini index given feature A is:

$$\mathrm{Gini}(D \mid A) = \sum_{i=1}^{m} \frac{|D_{a_i}|}{|D|} \mathrm{Gini}(D_{a_i}). \tag{1.56}$$

For feature selection, we choose the feature that minimizes the conditional Gini index:

$$A^\star = \operatorname*{argmin}_{A} \mathrm{Gini}(D \mid A). \tag{1.57}$$

Remark

For binary classification, the Gini index and entropy (scaled by $1/2$) are very similar functions of the class probability p. Both achieve their minimum at $p \in \{0, 1\}$ and maximum at $p = 1/2$.

For regression problems, where labels are continuous, we use a different criterion based on squared loss.

Definition 1.8 (L2 Loss Criterion)

For a regression problem, after splitting by feature A into subsets $D_1, \ldots, D_m$, define the mean label in each subset:

$$\bar{y}_{D_i} = \frac{1}{|D_i|} \sum_{j \in D_i} y_j. \tag{1.58}$$

The L2 loss criterion is:

$$L(D, A) = \sum_{i=1}^{m} \sum_{j \in D_i} (y_j - \bar{y}_{D_i})^2. \tag{1.59}$$

We choose the feature that minimizes this criterion:

$$A^\star = \operatorname*{argmin}_A L(D, A). \tag{1.60}$$

♣

This criterion measures the variance within each subset after splitting. A smaller L2 loss indicates purer subsets, where examples in each subset have labels close to their subset mean. The mean $\bar{y}_{D_i}$ serves as the prediction for examples in subset D_i.

1.1.5 Building Decision Trees: Greedy Algorithm

Given multiple feature selection criteria (information gain, information gain ratio, Gini index, or L2 loss), we can build a decision tree. The naive approach of enumerating all possible trees is computationally infeasible (super-exponential complexity). Instead, we use a *greedy algorithm*.

The greedy algorithm builds the tree recursively:

Input: Training set D, feature set $\mathcal{F}$.

Procedure:

1. **Choose a feature** $A \in \mathcal{F}$ according to a purity metric (e.g., information gain, information gain ratio, Gini index, or L2 loss).
2. **Partition** D into subsets $D_1, \ldots, D_m$ based on feature A's values $a_1, \ldots, a_m$.
3. **Recursively build subtrees** for each subset D_i using the remaining features $\mathcal{F} \setminus \{A\}$.
   Why remove A? Since all examples in D_i share the same value for feature A, further splitting by A would be redundant.
4. **Termination conditions:**
   - The feature set $\mathcal{F}$ is empty (no more features available).
   - The subset D_i is pure (contains only one class for classification, or has very low variance for regression).
   - Maximum depth is reached.
   - Purity improvement is below a threshold.
   - Number of samples in a node is below a minimum threshold.
5. When termination occurs, the node becomes a **leaf node** with label equal to the majority class (for classification) or the mean value (for regression) in that subset.

Prediction: For a test example, traverse the tree according to its feature values until reaching a leaf node, then use the leaf's label as the prediction.

Remark

The naive approach of enumerating all possible trees would require exploring all permutations of feature selection orders, leading to super-exponential complexity. The greedy algorithm reduces this to polynomial time by making locally optimal choices at each step, though it does not guarantee global optimality.

Remark

To prevent overfitting, common regularization techniques include:

- *Limiting the maximum depth of the tree.*
- *Setting a minimum number of samples required to split a node.*
- *Setting a minimum purity improvement threshold.*
- *Using a validation set to monitor performance and stop early.*

Remark

The greedy algorithm is heuristic: it makes locally optimal choices at each step but does not guarantee a globally optimal tree. Despite this, decision trees and their ensemble variants (Random Forests, Gradient Boosting) are among the most powerful methods in practice, especially for tabular data.

1.1.6 Continuous Features in Decision Trees

In the preceding discussion, we implicitly assumed that features are discrete, taking values in a finite set. For such a feature A with possible values $\{a_1, \ldots, a_m\}$, a split on A partitions the training set into m subsets $D_{a_1}, \ldots, D_{a_m}$.

In practice, many features are continuous. To handle a continuous feature A, we can discretize it by introducing *thresholds*. Let

$$\{a^{(1)}, \ldots, a^{(n)}\} \tag{1.61}$$

be the distinct values of A observed in the training set (for some feature dimension). Sort them so that

$$a^{(1)} < a^{(2)} < \cdots < a^{(n)}. \tag{1.62}$$

Candidate thresholds can then be placed between successive values:

$$\tau_j = \frac{a^{(j)} + a^{(j+1)}}{2}, \qquad j = 1, \ldots, n-1. \tag{1.63}$$

Each threshold τ_j defines a binary split

$$A \leq \tau_j \quad \text{vs.} \quad A > \tau_j, \tag{1.64}$$

which we can evaluate using any of our usual criteria (information gain, Gini index, squared loss for regression, etc.). By scanning over all candidate thresholds and choosing the best one according to the purity measure, the tree handles continuous features while remaining axis-aligned.

At prediction time, a test example with continuous feature value $A(x)$ is routed through the tree by comparing $A(x)$ with the learned thresholds at each internal node, just as if the feature had been discrete.

1.2 From Single Trees to Ensembles

So far, we have seen how to build and select a *single* decision tree, by choosing features that maximize information gain at each split. In practice, however, a single tree often suffers from two issues:

- **High variance.** Small changes in the training data may lead to very different trees, especially when the tree is grown

deep and fits the training set closely.

- **Limited accuracy.** A single tree is easy to interpret, but its predictive performance may lag behind more powerful models such as kernel methods or neural networks.

Ensemble learning addresses these issues by combining the predictions of many base learners (often decision trees) to form a stronger model.

Definition 1.9 (Ensemble Learning)

Ensemble learning constructs a predictor by aggregating a collection of base learners (also called weak learners*). Given base predictors $f_1, \dots, f_M$, the ensemble predictor takes the form*

$$F(x) = \begin{cases} \text{majority vote}\big(f_1(x), \dots, f_M(x)\big), & \textit{(classification)}, \\ \dfrac{1}{M} \sum_{m=1}^{M} f_m(x), & \textit{(regression)}. \end{cases} \tag{1.65}$$

The key idea is that even if each f_m is only moderately accurate, their combination can be significantly more accurate and more robust. ♣

1.2.1 Bias–Variance Decomposition

Consider a supervised learning setting with training dataset D, drawn from some unknown data-generating distribution. Let x denote a test input with true label y, and let

$$f(x; D) \tag{1.66}$$

denote the prediction of a learning algorithm trained on D when evaluated at x. Under squared loss, the prediction error at (x, y) for a given training set D is

$$\big(f(x; D) - y\big)^2. \tag{1.67}$$

Since D is random, it is natural to measure performance by averaging over the random draw of D. We therefore consider

$$\mathbb{E}_D\Big[\big(f(x; D) - y\big)^2\Big], \tag{1.68}$$

which quantifies the expected squared error at x, averaged over the randomness in the training data.

Define the *average prediction* at x as

$$\bar{f}(x) := \mathbb{E}_D\big[f(x; D)\big], \tag{1.69}$$

that is, the expected prediction we would obtain if we could repeatedly sample training sets D from the underlying distribution and retrain the model. Insert and subtract $\bar{f}(x)$ inside the square:

$$\mathbb{E}_D\Big[\big(f(x; D) - y\big)^2\Big] = \mathbb{E}_D\Big[\big(f(x; D) - \bar{f}(x) + \bar{f}(x) - y\big)^2\Big] \tag{1.70}$$

$$= \mathbb{E}_D\Big[\big(f(x; D) - \bar{f}(x)\big)^2\Big] + \big(\bar{f}(x) - y\big)^2 \tag{1.71}$$

$$+ 2\,\mathbb{E}_D\Big[\big(f(x; D) - \bar{f}(x)\big)\big(\bar{f}(x) - y\big)\Big]. \tag{1.72}$$

The cross term vanishes because

$$\mathbb{E}_D[f(x; D) - \bar{f}(x)] = \mathbb{E}_D[f(x; D)] - \bar{f}(x) = 0, \tag{1.73}$$

and $\bar{f}(x) - y$ does not depend on D. Hence

$$\mathbb{E}_D\Big[\big(f(x; D) - y\big)^2\Big] = \underbrace{\mathbb{E}_D\Big[\big(f(x; D) - \bar{f}(x)\big)^2\Big]}_{\text{variance at } x} + \underbrace{\big(\bar{f}(x) - y\big)^2}_{\text{bias}^2 \text{ at } x}. \tag{1.74}$$

Definition 1.10 (Bias and variance at a test point)

For a fixed test input x with true label y, the variance *and* (squared) bias *of a learning algorithm are defined as*

$$\mathrm{Var}_D\left[f(x;D)\right] = \mathbb{E}_D\left[\left(f(x;D) - \bar{f}(x)\right)^2\right], \tag{1.75}$$

$$\mathrm{Bias}^2(x) = \left(\bar{f}(x) - y\right)^2. \tag{1.76}$$

♣

Intuitively:

- The variance term measures how sensitive the predictor $f(x;D)$ is to fluctuations in the training data: for different draws of D, predictions at x may vary significantly around their mean $\bar{f}(x)$.
- The bias term measures how far, on average, the prediction is from the true label y. Even if we could train on infinitely many different datasets, the average predictor $\bar{f}(x)$ might still systematically miss the true target due to limited model capacity or misspecification.

High variance typically arises from *overfitting* particular training sets, while high bias indicates that the model class itself is too simple to capture the underlying relationship, even when averaged over many datasets.

Example 1.3 – Linear model on nonlinear data

Suppose the data arise from a quadratic relationship, but we insist on fitting a linear model. Even if we repeatedly resample training sets and average the resulting fitted lines, the averaged predictor $\bar{f}(x)$ remains linear and cannot match the true quadratic curve everywhere. This manifests as a persistent bias term.

Bias and variance suggest complementary strategies:

- To reduce **variance**, we can increase the training set size or average multiple diverse models (as in bagging and random forests).
- To reduce **bias**, we can increase model capacity, e.g. by using more flexible hypothesis classes or boosting weak learners.

Two classical ensemble strategies are:

- *Bagging* (Bootstrap Aggregating): build many base learners independently on randomized versions of the data, and average or vote.
- *Boosting*: build base learners sequentially, each one focusing on correcting the mistakes of the previous ones.

In modern machine learning practice, decision-tree ensembles based on bagging and boosting are among the most powerful off-the-shelf methods, with famous examples including Random Forests, Gradient Boosting Trees, and XGBoost.

1.2.2 Bagging: Bootstrap Aggregating

Bagging is designed to reduce the variance of an unstable base learner (such as a deep decision tree) by averaging many independently trained copies.

Definition 1.11 (Bagging)

Let D be a training dataset of size n, and let $\mathcal{A}$ be a base learning algorithm that outputs a predictor $f = \mathcal{A}(D)$. Bagging constructs M bootstrap datasets $D^{(1)}, \ldots, D^{(M)}$, where each $D^{(m)}$ is obtained by sampling n points with replacement *from D.*
On each bootstrap dataset $D^{(m)}$, we train a base learner

$$f_m = \mathcal{A}\left(D^{(m)}\right), \qquad m = 1, \ldots, M. \tag{1.77}$$

The bagged ensemble predictor is

$$F_{\text{bag}}(x) = \begin{cases} \operatorname{sign}\Big(\frac{1}{M}\sum_{m=1}^{M} f_m(x)\Big), & \textit{classification}, \\ \frac{1}{M}\sum_{m=1}^{M} f_m(x), & \textit{regression}. \end{cases} \tag{1.78}$$

♣

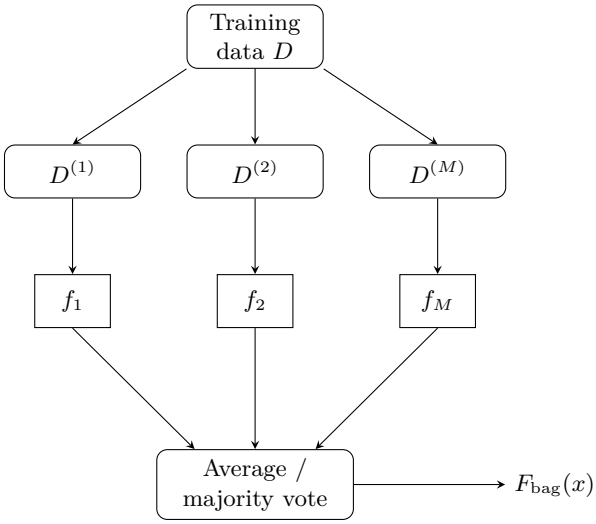
Intuitively, each bootstrap sample $D^{(m)}$ can be viewed as a noisy version of the original dataset. The base learners f_m will differ from one another, and averaging their predictions cancels out part of the randomness.

Remark

Consider a simplified setting where we average M identically distributed base predictors $f_1, \ldots, f_M$ for regression, each with variance σ^2 and pairwise correlation ρ. Then the variance of the averaged predictor is

$$\operatorname{Var}\big(F_{\text{bag}}(x)\big) = \frac{1}{M^2}\sum_{m=1}^{M}\sum_{m'=1}^{M} \operatorname{Cov}\big(f_m(x), f_{m'}(x)\big) \approx \sigma^2\left(\rho + \frac{1-\rho}{M}\right). \tag{1.79}$$

As M grows, the second term $(1-\rho)/M$ vanishes, so the variance is dominated by $\rho\sigma^2$. Thus, bagging is most effective when base learners are accurate but have high variance and are not too strongly correlated.



1.2.3 Random Forests

Random Forests specialize bagging to decision trees, and further inject randomness at the *feature* level to decorrelate the trees.

Definition 1.12 (Random Forest)

A Random Forest is an ensemble of decision trees trained with two sources of randomness:

1. ***Bootstrap sampling of data.*** *For each tree m, sample a bootstrap dataset $D^{(m)}$ from D.*
2. ***Random feature selection at each split.*** *When splitting a node, instead of considering all features, randomly select a subset $\mathcal{F}$ of features (of size d_{sub}), and choose the best split only among features in $\mathcal{F}$.*

The final prediction aggregates all trees by majority vote or averaging, as in bagging.

♣

The additional randomness in feature selection has two important effects:

- It reduces the correlation between different trees, which strengthens the variance reduction effect of averaging.

- It forces each tree to explore different feature combinations, sometimes discovering useful patterns that a single greedy tree might miss.

In practice, Random Forests are strong general-purpose models with relatively few hyperparameters. Common choices include:

- the number of trees M (often in the hundreds),
- the maximum depth of each tree,
- the number of features d_{sub} considered at each split (e.g., $\sqrt{d}$ for classification, where d is the total number of features).

Remark

In each bootstrap sample $D^{(m)}$, roughly a fraction $1 - 1/e \approx 0.63$ of the original training points are included, and the remaining points are left out. For any training example, we can average the predictions of all trees that did not *see this example during training; this is called the* out-of-bag *prediction. Aggregating these predictions over the training set provides an internal estimate of the generalization error, without using a separate validation set.*

1.2.4 Boosting

While bagging focuses on variance reduction by averaging many independent learners, boosting builds an ensemble *sequentially*. Each new learner tries to correct the mistakes of the current ensemble, effectively turning a collection of weak learners into a strong one.

Definition 1.13 (Boosting)

Given a training set $D = \{(x_i, y_i)\}_{i=1}^n$, boosting maintains an ensemble predictor

$$F_0(x) \equiv 0, \qquad F_M(x) = \sum_{m=1}^{M} \alpha_m f_m(x), \tag{1.80}$$

where each f_m is a base learner (often a shallow tree) and the weights α_m control their influence.
At iteration m, boosting chooses f_m to focus on the current residual errors or misclassified examples of F_{m-1}, and then updates the ensemble to F_m.

♣

One classical example is AdaBoost for binary classification. It maintains a distribution of weights over training samples, gives higher weights to misclassified points, and fits a new weak learner to this reweighted dataset at each iteration.

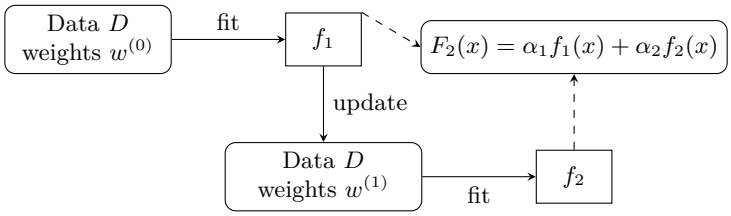
Remark

- *Bagging trains base learners* in parallel *on resampled datasets and primarily reduces variance.*
- *Boosting trains base learners* sequentially, *focusing on difficult samples, and can reduce both bias and variance, but is also more prone to overfitting if not regularized (e.g., via tree depth, learning rate, or early stopping).*

Modern implementations such as Gradient Boosting Trees and XGBoost can be viewed as performing a form of functional gradient descent in function space, where each tree fits the negative gradient of the loss with respect to the current ensemble prediction.

AdaBoost can be derived as stagewise minimization of the exponential loss

$$\mathcal{L}(F) = \sum_{i=1}^{n} \exp\big(-y_i F(x_i)\big), \qquad y_i \in \{-1, +1\}.$$



Suppose at round m we update

$$F_m(x) = F_{m-1}(x) + \alpha h(x), \qquad h(x) \in \{-1, +1\}.$$

Then

$$\begin{aligned}\mathcal{L}(F_m) &= \sum_{i=1}^{n} \exp\big(-y_i(F_{m-1}(x_i) + \alpha h(x_i))\big) \\ &= \sum_{i=1}^{n} \exp\big(-y_i F_{m-1}(x_i)\big) \exp\big(-\alpha y_i h(x_i)\big). \end{aligned} \tag{1.81}$$

Define the normalized weights (sigmoid)

$$w_i^{(m)} = \frac{\exp\big(-y_i F_{m-1}(x_i)\big)}{\sum_{j=1}^{n} \exp\big(-y_j F_{m-1}(x_j)\big)}$$

Minimizing $\mathcal{L}(F_m)$ w.r.t. α is equivalent to minimizing

$$Z(\alpha) := \sum_{i=1}^{n} w_i^{(m)} \exp\big(-\alpha y_i h(x_i)\big).$$

Let the weighted error of h under $w^{(m)}$ be

$$\varepsilon = \sum_{i=1}^{n} w_i^{(m)} \mathbb{1}\big(h(x_i) \neq y_i\big).$$

Note that $y_i h(x_i) = +1$ if the point is correctly classified and $y_i h(x_i) = -1$ otherwise. Splitting the sum into correct vs. incorrect sets,

$$Z(\alpha) = (1 - \varepsilon)e^{-\alpha} + \varepsilon e^{\alpha}.$$

Differentiate and set to zero:

$$\frac{\mathrm{d}Z}{\mathrm{d}\alpha} = -(1 - \varepsilon)e^{-\alpha} + \varepsilon e^{\alpha} = 0 \quad \Longrightarrow \quad e^{2\alpha} = \frac{1 - \varepsilon}{\varepsilon}.$$

Hence the minimizer is

$$\alpha^{\star} = \frac{1}{2} \log \frac{1 - \varepsilon}{\varepsilon}.$$

Substituting $\alpha^{\star}$ back gives the minimum

$$Z(\alpha^{\star}) = 2\sqrt{\varepsilon(1 - \varepsilon)},$$

which is < 1 whenever $\varepsilon < \frac{1}{2}$. This explains why AdaBoost requires each weak learner to perform slightly better than random guessing.

Example 1.4 – AdaBoost

Let's use AdaBoost to solve a one-dimensional binary classification problem.
Consider five training samples with labels $y_i \in \{-1, +1\}$:

i	1	2	3	4	5
x_i	1	2	3	4	5
y_i	+1	+1	−1	−1	+1

Solution

AdaBoost starts with uniform sample weights

$$w_i^{(1)} = \frac{1}{5}, \qquad i = 1, \ldots, 5.$$

A decision stump is parameterized by a threshold t and a polarity $s \in \{+1, -1\}$:

$$h_{t,s}(x) = \begin{cases} s, & x < t, \\ -s, & x \geq t. \end{cases}$$

Since the data are one-dimensional and sorted, candidate thresholds are the midpoints

$$t \in \{1.5,\ 2.5,\ 3.5,\ 4.5\}.$$

We evaluate the weighted classification error

$$\varepsilon_1(t, s) = \sum_{i=1}^{5} w_i^{(1)} \mathbb{1}\big(h_{t,s}(x_i) \neq y_i\big)$$

for all (t, s) and select the minimizer. The optimal choice is $(t_1, s_1) = (2.5, +1)$, i.e.

$$h_1(x) = \begin{cases} +1, & x < 2.5, \\ -1, & x \geq 2.5, \end{cases}$$

which misclassifies only sample $i = 5$. Hence

$$\varepsilon_1 = \frac{1}{5}.$$

AdaBoost assigns the weak learner a coefficient

$$\alpha_1 = \frac{1}{2} \log \frac{1 - \varepsilon_1}{\varepsilon_1} = \frac{1}{2} \log 4 = \log 2.$$

Weights are updated according to

$$w_i^{(2)} = \frac{w_i^{(1)} \exp\big(-\alpha_1 y_i h_1(x_i)\big)}{Z_1},$$

where Z_1 normalizes the weights to sum to 1. Explicitly,

$$w^{(2)} = (0.125,\ 0.125,\ 0.125,\ 0.125,\ 0.5).$$

The misclassified point x_5 now carries half of the total weight.
Using weights $w^{(2)}$, we again evaluate all candidate stumps. The minimum weighted error is attained at $(t_2, s_2) = (4.5, -1)$:

$$h_2(x) = \begin{cases} -1, & x < 4.5, \\ +1, & x \geq 4.5, \end{cases}$$

which misclassifies only samples $i = 1, 2$. Therefore,

$$\varepsilon_2 = w_1^{(2)} + w_2^{(2)} = 0.25.$$

$$\alpha_2 = \frac{1}{2} \log \frac{1 - \varepsilon_2}{\varepsilon_2} = \frac{1}{2} \log 3.$$

After two iterations, the AdaBoost classifier is

$$F_2(x) = \alpha_1 h_1(x) + \alpha_2 h_2(x), \qquad \hat{y} = \operatorname{sign}\big(F_2(x)\big).$$

We now compute the final predictions of F_2 on the five training samples. First evaluate the two stumps:

i	1	2	3	4	5
x_i	1	2	3	4	5
$h_1(x_i)$	+1	+1	−1	−1	−1
$h_2(x_i)$	−1	−1	−1	−1	+1

Hence,

$$F_2(x_1) = \alpha_1 - \alpha_2 = \log 2 - \frac{1}{2}\log 3 = \frac{1}{2}\log\frac{4}{3} > 0, \tag{1.82}$$

$$F_2(x_2) = \alpha_1 - \alpha_2 = \frac{1}{2}\log\frac{4}{3} > 0, \tag{1.83}$$

$$F_2(x_3) = -\alpha_1 - \alpha_2 < 0, \tag{1.84}$$

$$F_2(x_4) = -\alpha_1 - \alpha_2 < 0, \tag{1.85}$$

$$F_2(x_5) = -\alpha_1 + \alpha_2 = \frac{1}{2}\log 3 - \log 2 = \frac{1}{2}\log\frac{3}{4} < 0. \tag{1.86}$$

Therefore,

$$\hat{y}(x_1) = +1, \quad \hat{y}(x_2) = +1, \quad \hat{y}(x_3) = -1, \quad \hat{y}(x_4) = -1, \quad \hat{y}(x_5) = -1,$$

The last sample is still misclassified, but with a smaller margin*, since* $F_2(x_5) = -\alpha_1 + \alpha_2$ *is closer to zero than* $F_1(x_5) = -\alpha_1$.

Note

From a geometric perspective, both Random Forests and boosting-based tree ensembles can be seen as constructing a complex, highly nonlinear decision boundary by patching together many simple, axis-aligned splits. Whereas a single tree corresponds to a small set of such partitions, an ensemble can carve out increasingly intricate decision regions, often achieving state-of-the-art performance on tabular data.